Artificial Intelligence

Lecture 14

Artificial Intelligence

- Agents and Intelligence
- Approaches in AI
- State Graphs and Search Trees
- Heuristics
- Neural Networks
- Natural Language Processing

Intelligent Agents

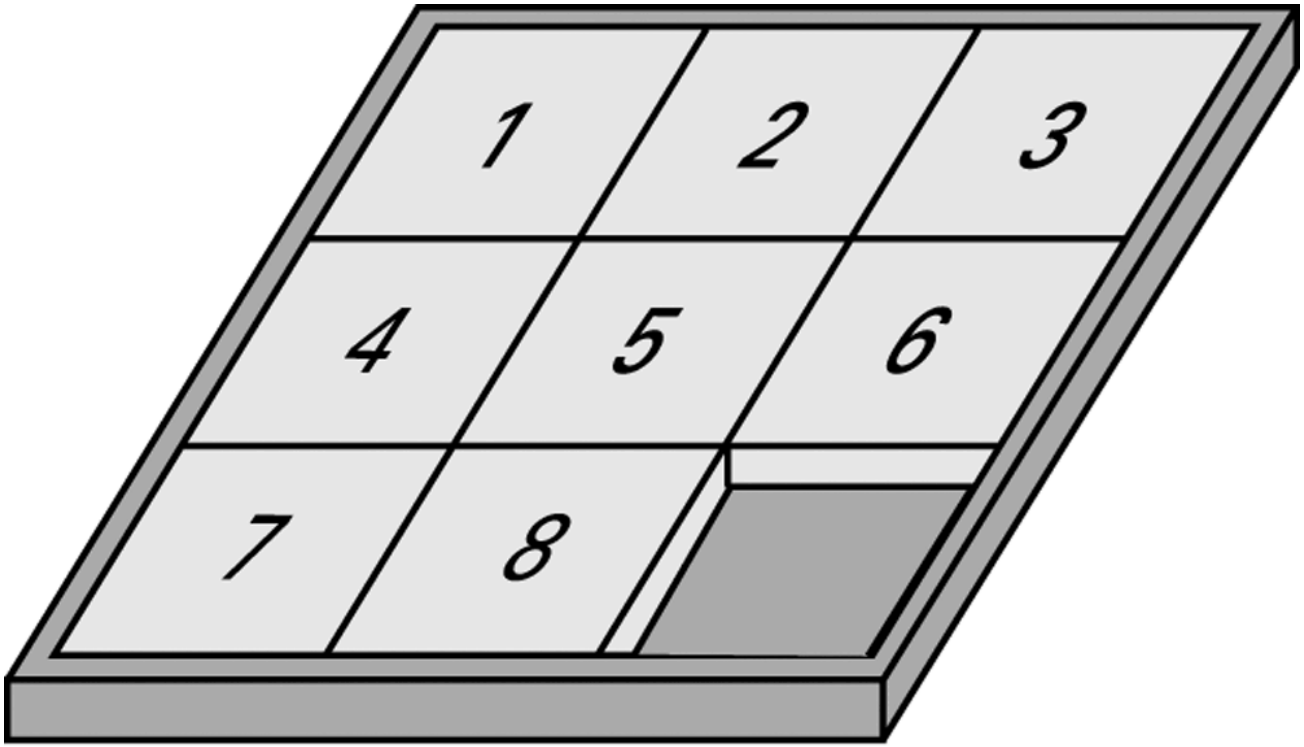
- **Agent:** A "device" that responds to stimuli from its environment
  - Sensors
  - Actuators
- Much of the research in artificial intelligence can be viewed in the context of building agents that behave intelligently

Herbert:
The Soda Can Collecting Robot
(Connell, Brooks, Ning, 1988)
http://cyberneticzoo.com/?p=5516

Levels of Intelligent Behavior

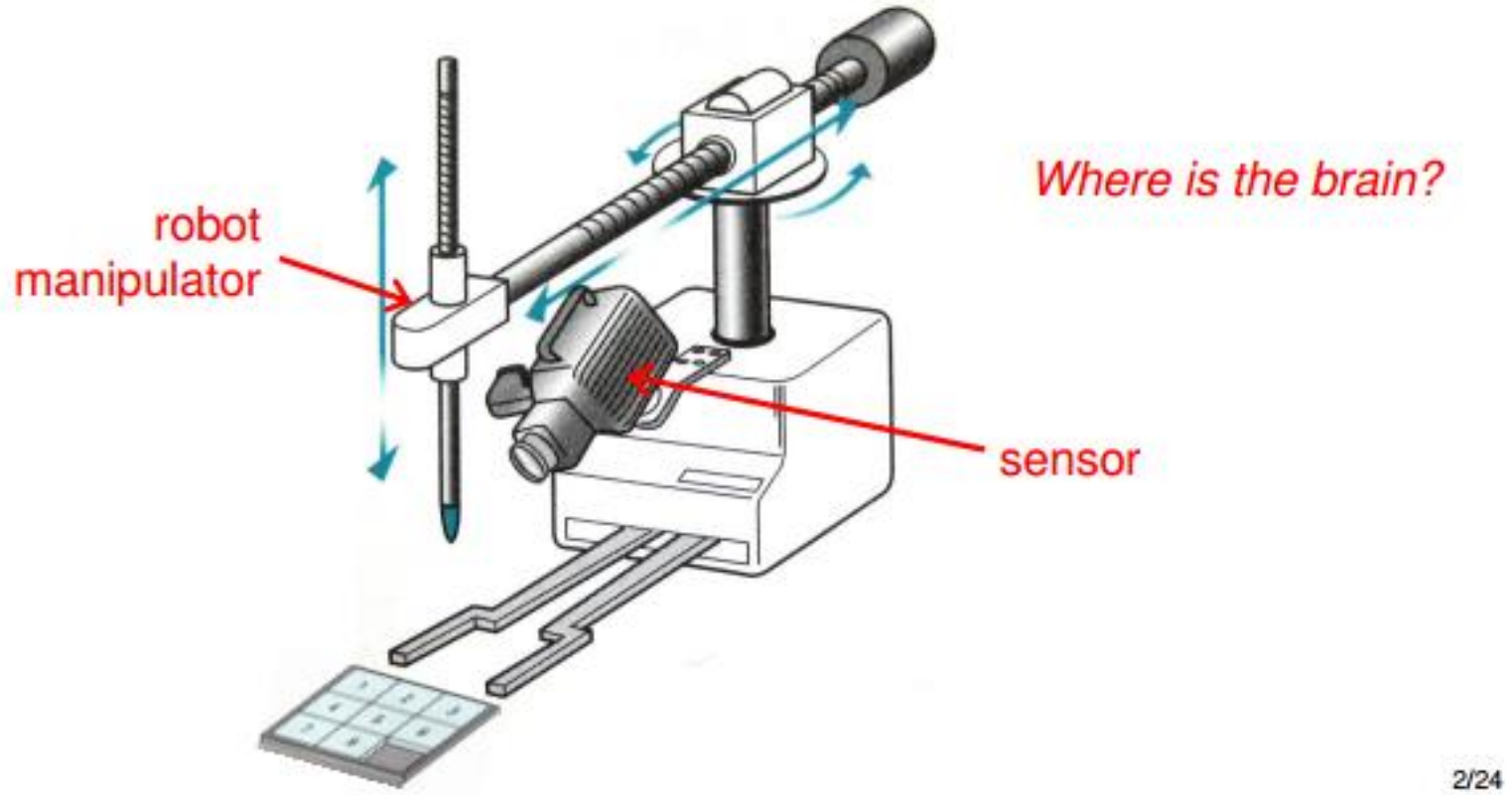
- Reflex: actions are predetermined responses to the input data
- More intelligent behavior requires knowledge of the environment and involves such activities as:
  - Goal seeking
  - Learning

The eight-puzzle in its solved configuration



Our puzzle-solving machine

In AI, researchers try to build a device (an agent) that can sense-and-change its environment



Techniques for Understanding Images

- Template matching
- Image processing
  - edge enhancement
  - region finding
  - smoothing
- Image analysis

Approaches to Research in Artificial Intelligence

- Performance-oriented
  - Researcher tries to maximize the performance of the agents; the techniques used may not be "intelligent" by nature, but are effective in producing "intelligent" results
- Simulation-oriented
  - Researcher tries to derive theories about how a biological agent produce intelligent responses to the environment and try to build an artificial agent that use the theories to simulate the behaviors

Components of a Production Systems

1. Collection of states
   - Start (or initial) state
   - Goal state (or states)
2. Collection of productions: rules or moves
   - Each production may have preconditions
3. Control system: decides which production to apply next

Reasoning by Searching

- **State Graph:** All states and productions
- **Search Tree:** A record of state transitions explored while searching for a goal state
  - Breadth-first search
  - Depth-first search

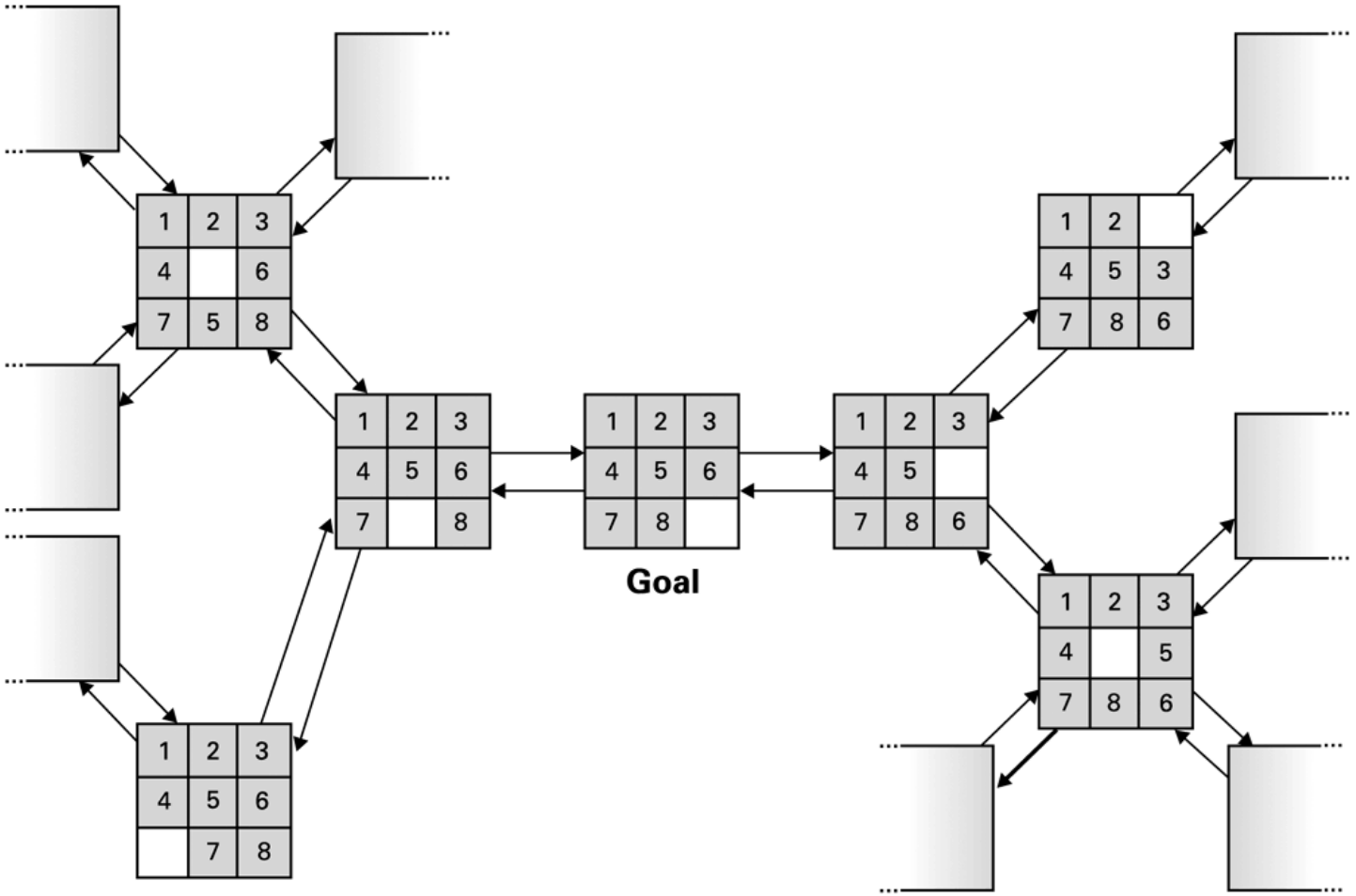
Depth First Search

```
def dfs(graph, start, target):
    visited, stack = set(), [start]
    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            print("visited %d" % vertex)
            stack.extend(graph[vertex] - visited)
        if vertex == target:
            break
    # return visited
```

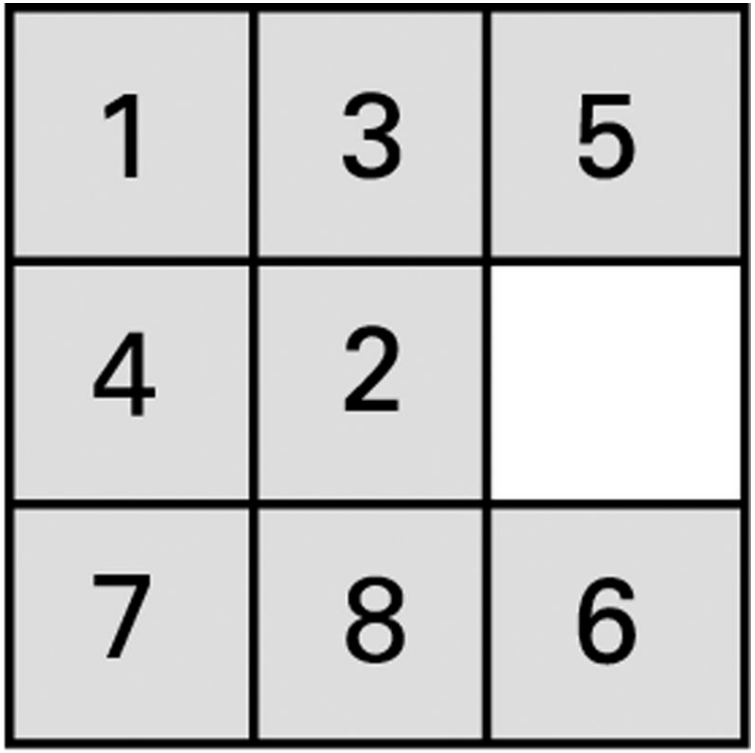
Breadth First Search

```
def bfs(graph, start, target):
    visited, queue = set(), [start]
    while queue:
        vertex = queue.pop(0)
        if vertex not in visited:
            visited.add(vertex)
            print("visited %d" % vertex)
            queue.extend(graph[vertex] - visited)
        if vertex == target:
            break
    # return visited
```

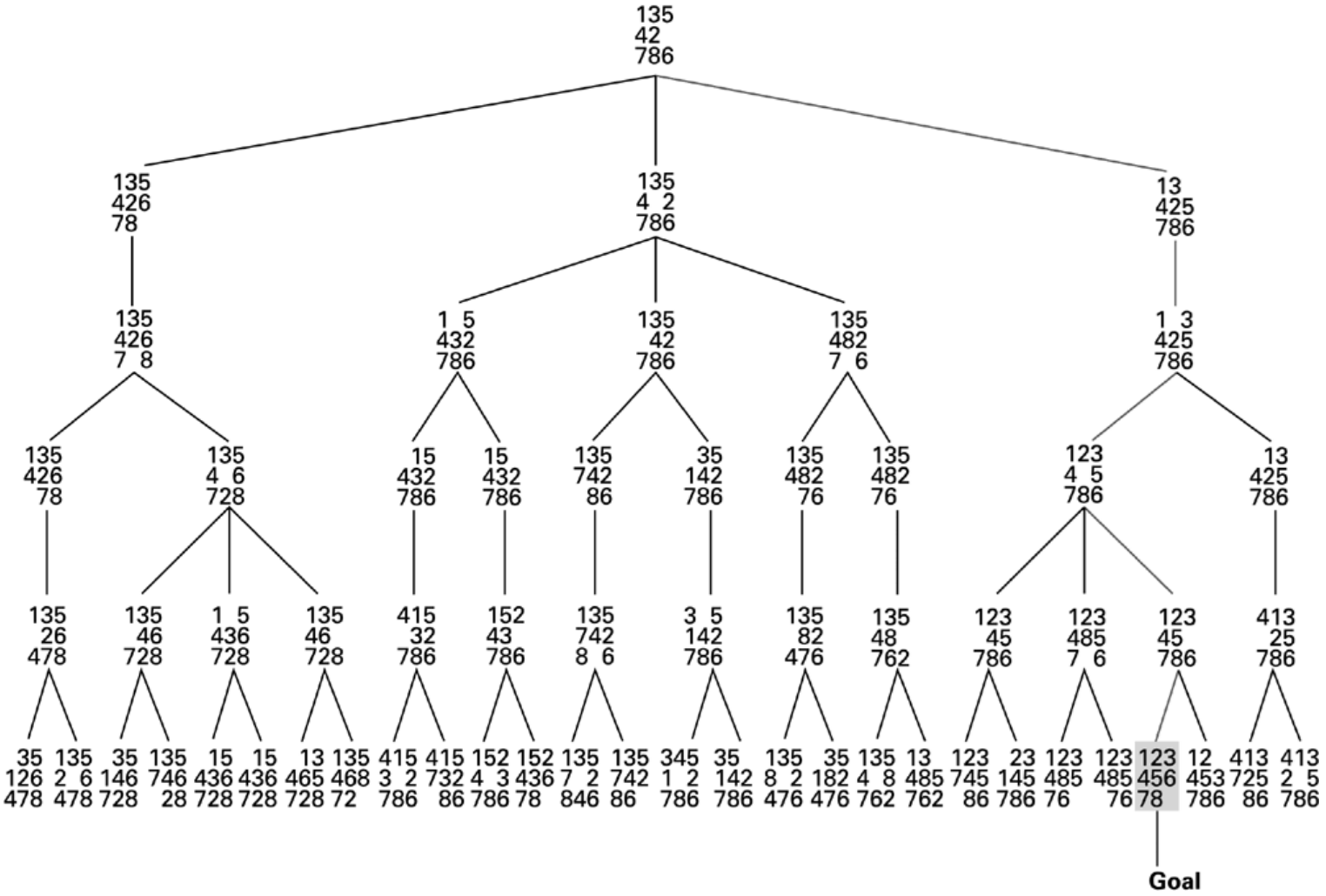
A small portion of the eight-puzzle's state graph



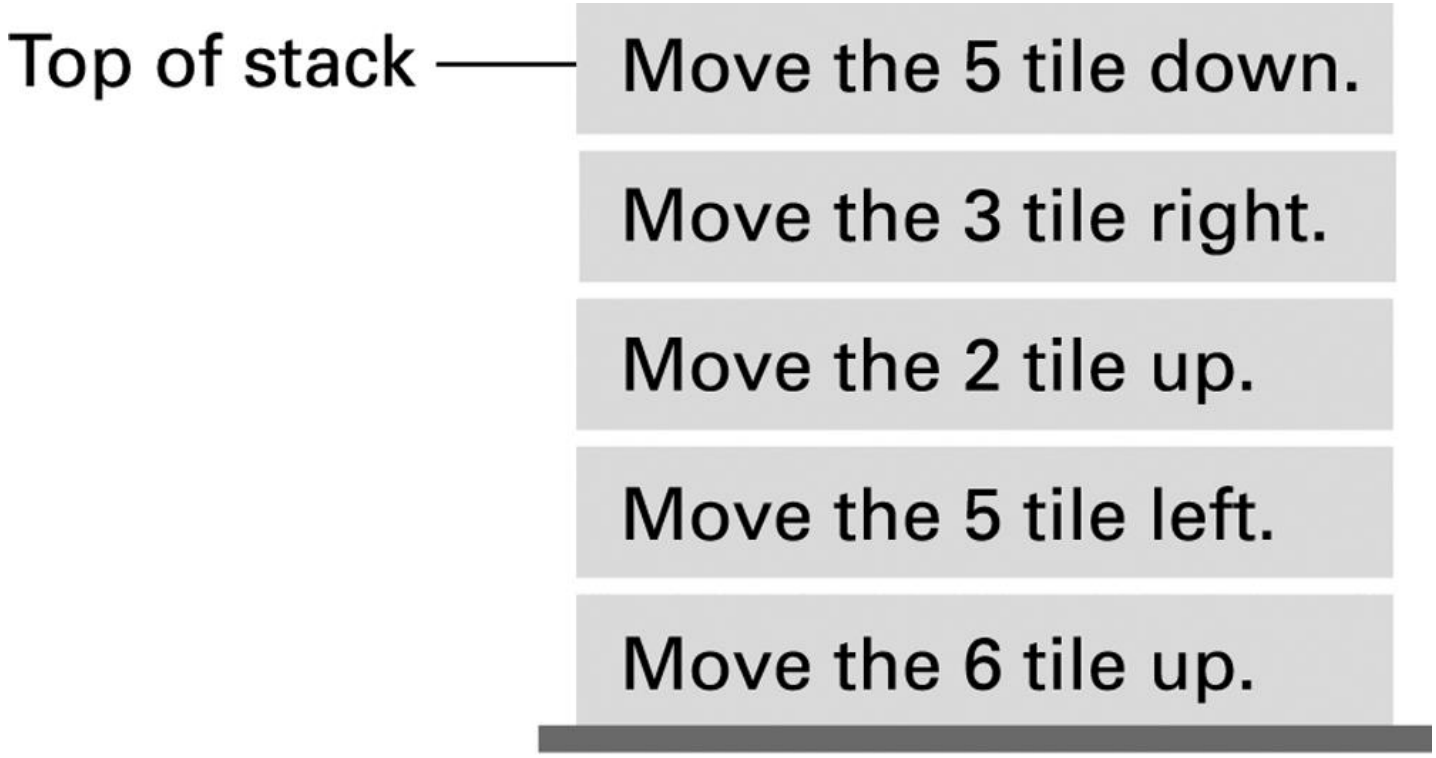
An unsolved eight-puzzle



A sample search tree



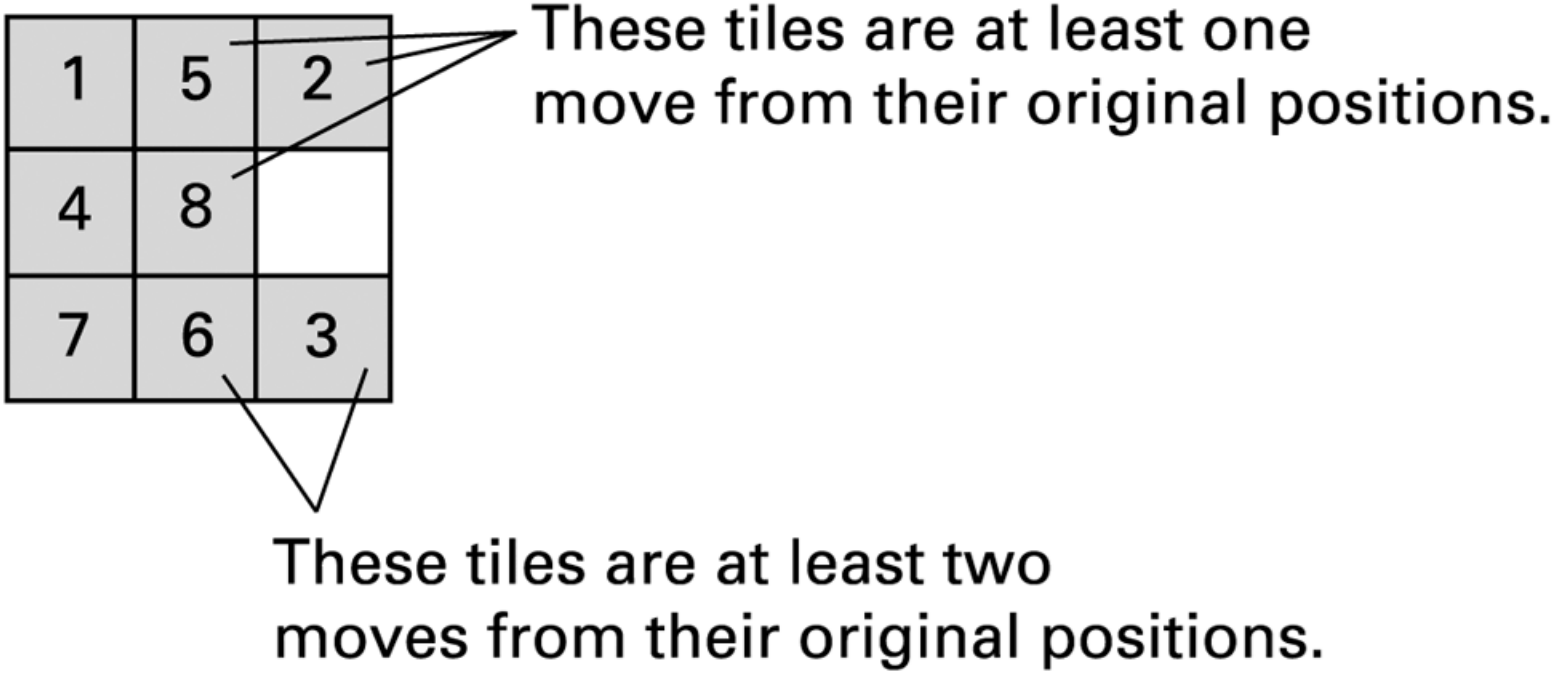
Productions stacked for later execution



Heuristic Strategies

- **Heuristic:** A “rule of thumb” for making decisions
- Requirements for good heuristics
  - Must be easier to compute than a complete solution
  - Must provide a reasonable estimate of proximity to a goal

An unsolved weight-puzzle



Heuristic search algorithm (sketch)

- Establish the start node of the state graph as the root of the search tree and record its heuristic value.

while (the goal node has not been reached):

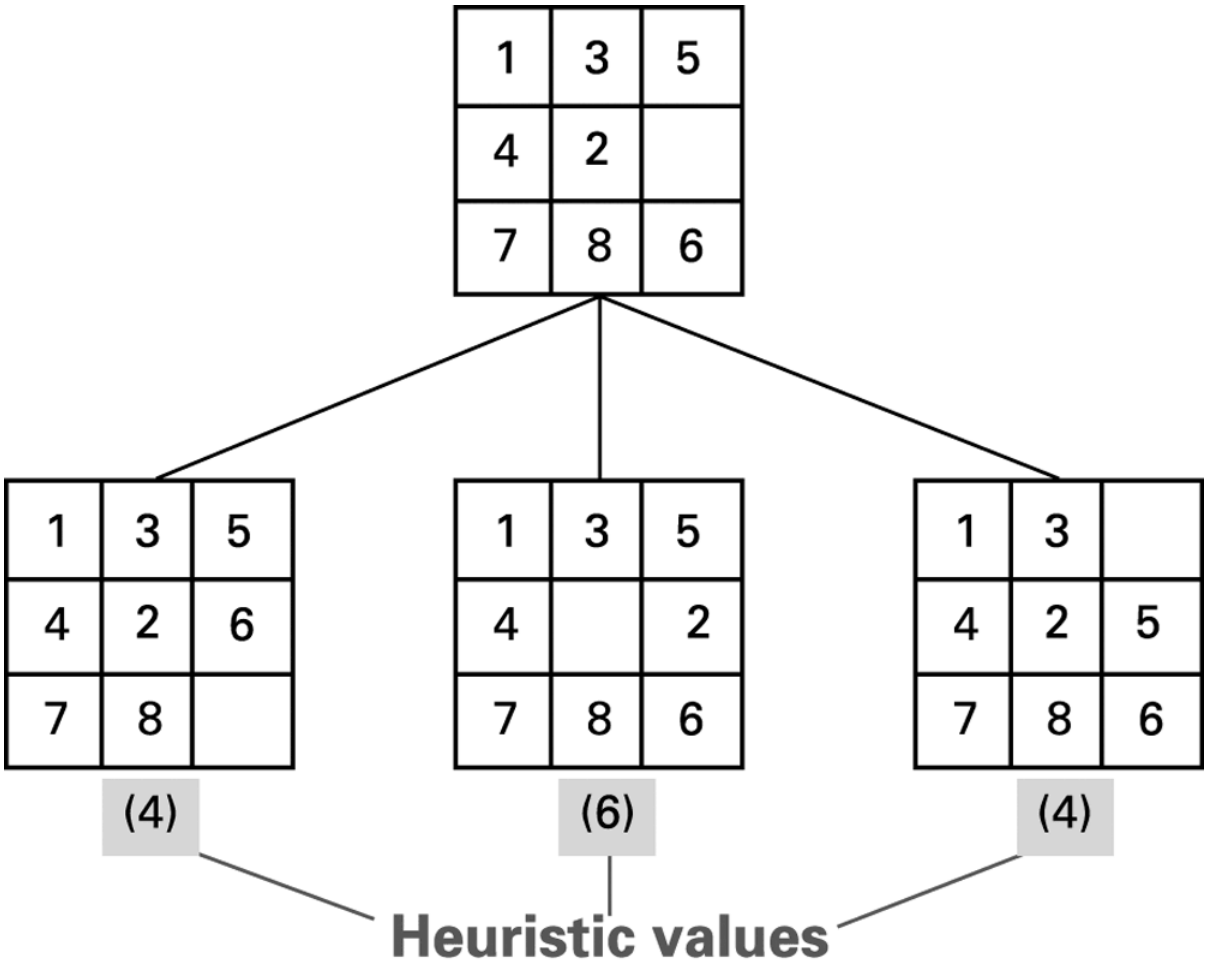
  - Select the leftmost leaf node with the smallest heuristic value of all leaf nodes.
  - To this selected node attach as children those nodes that can be reached by a single production.
  - Record the heuristic of each of these new nodes next to the node in the search tree.

- Traverse the search tree from the goal node up to the root, pushing the production associated with each arc traversed onto a stack.
- Solve the original problem by executing the productions as they are popped off the stack.

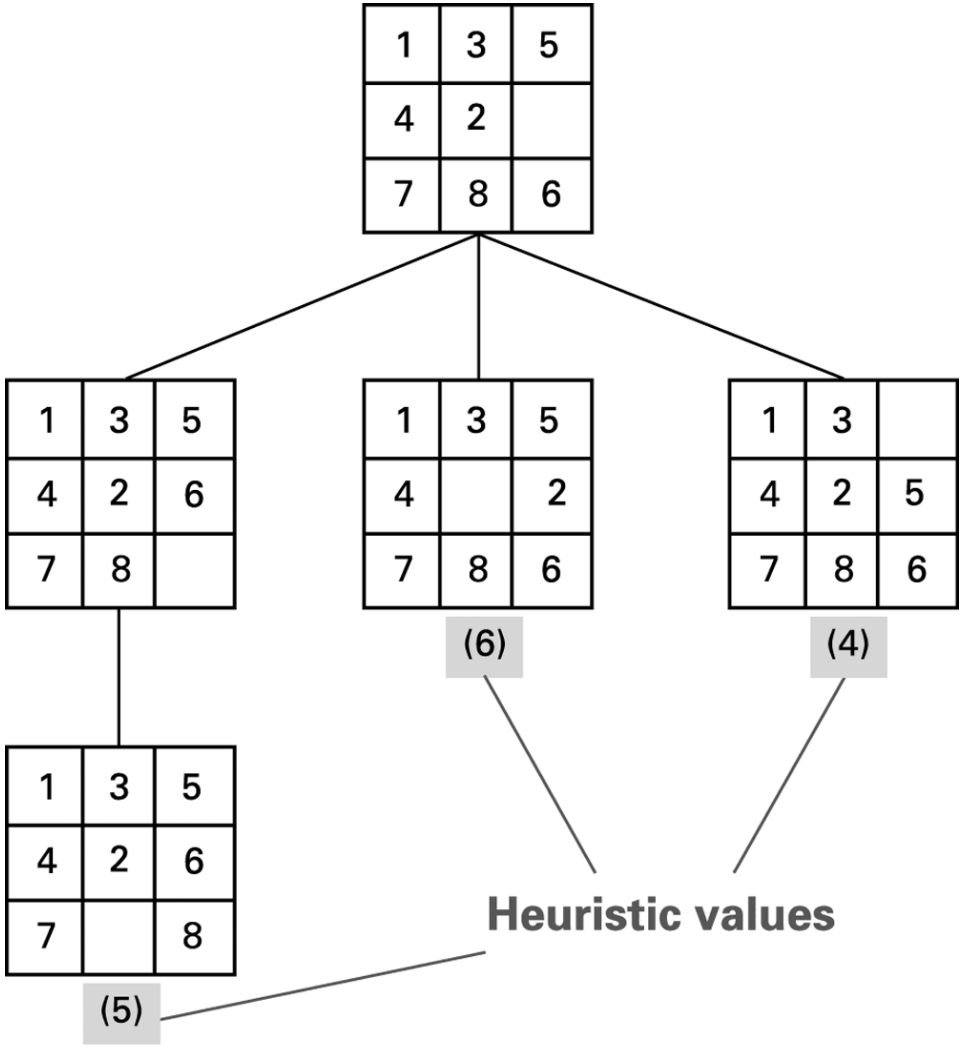
Heuristic Search

```
def heuristic(graph, start, target, h):
    start_tuple=(h(target,start), start)
    visited, pqueue = set(), [start_tuple]
    while pqueue:
        vertex_tuple = heappop(pqueue)
        vertex = vertex_tuple[1]
        if vertex not in visited:
            visited.add(vertex)
            print("visited %d, heuristic %d" % (vertex, vertex_tuple[0]))
            for v in graph[vertex] - visited:
                v_tuple = (h(target, v), v)
                heappush(pqueue, v_tuple)
        if vertex == target:
            break
    # return visited
```

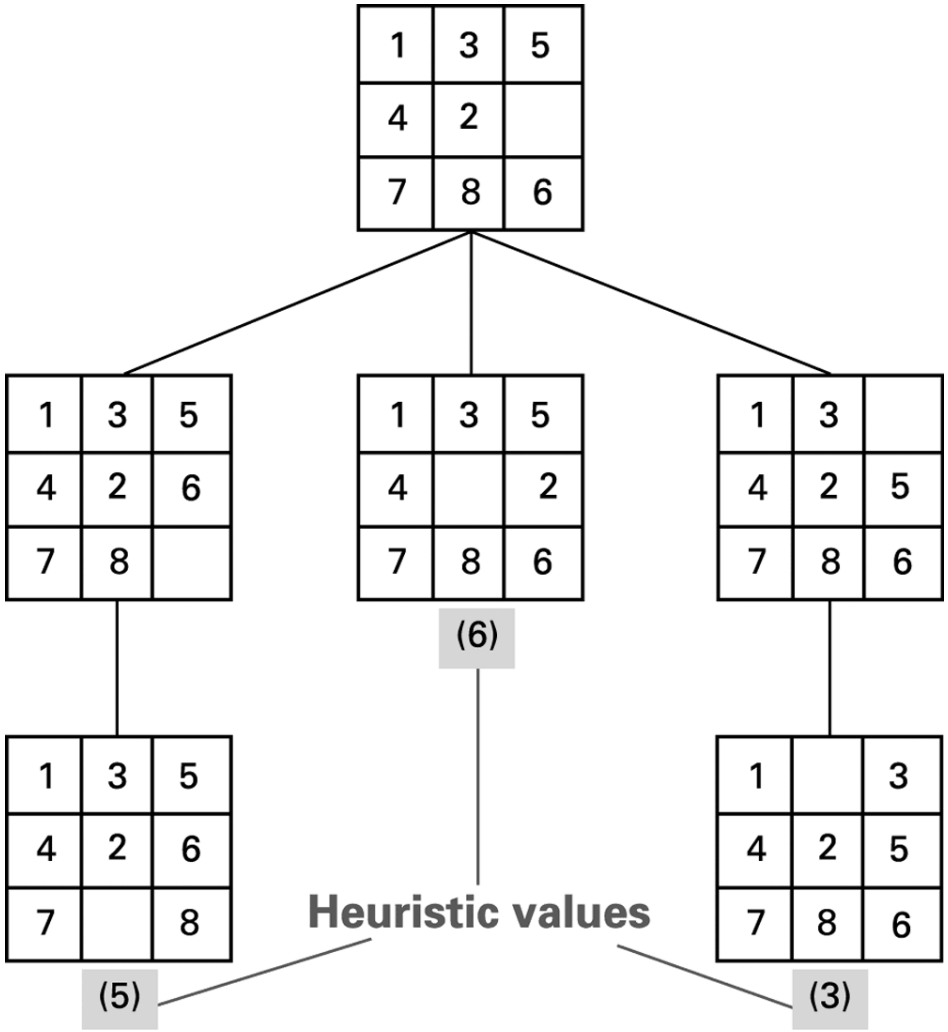
The beginnings of our heuristic search



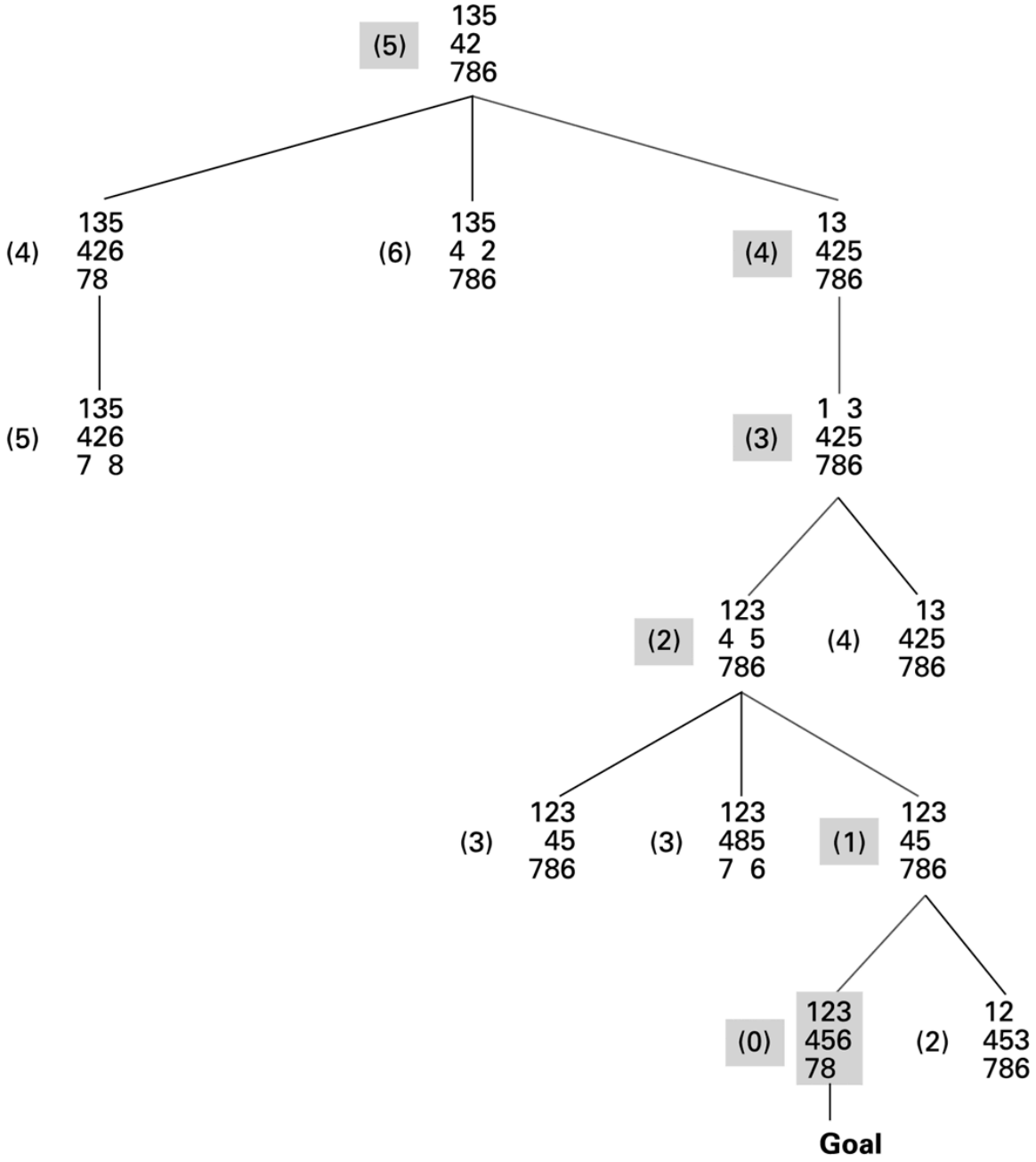
The search tree after two passes



The search tree after three passes



The complete search tree formed by our heuristic system

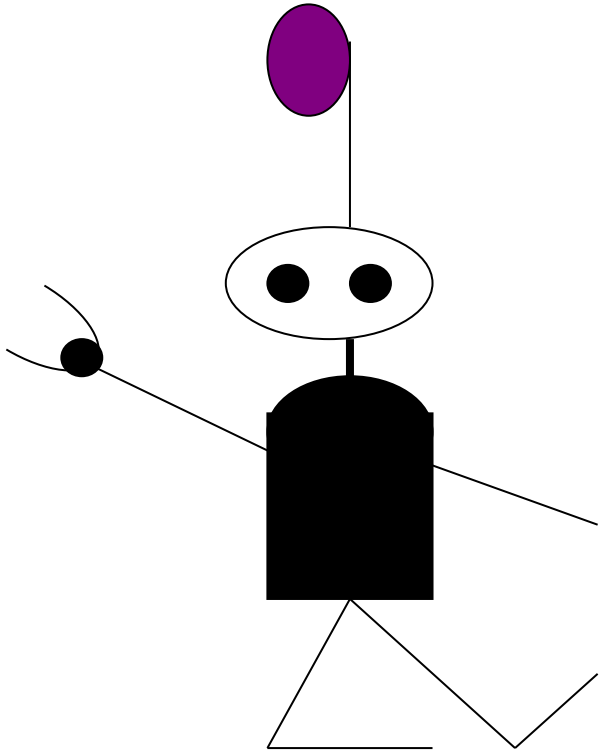


Genetic Algorithms

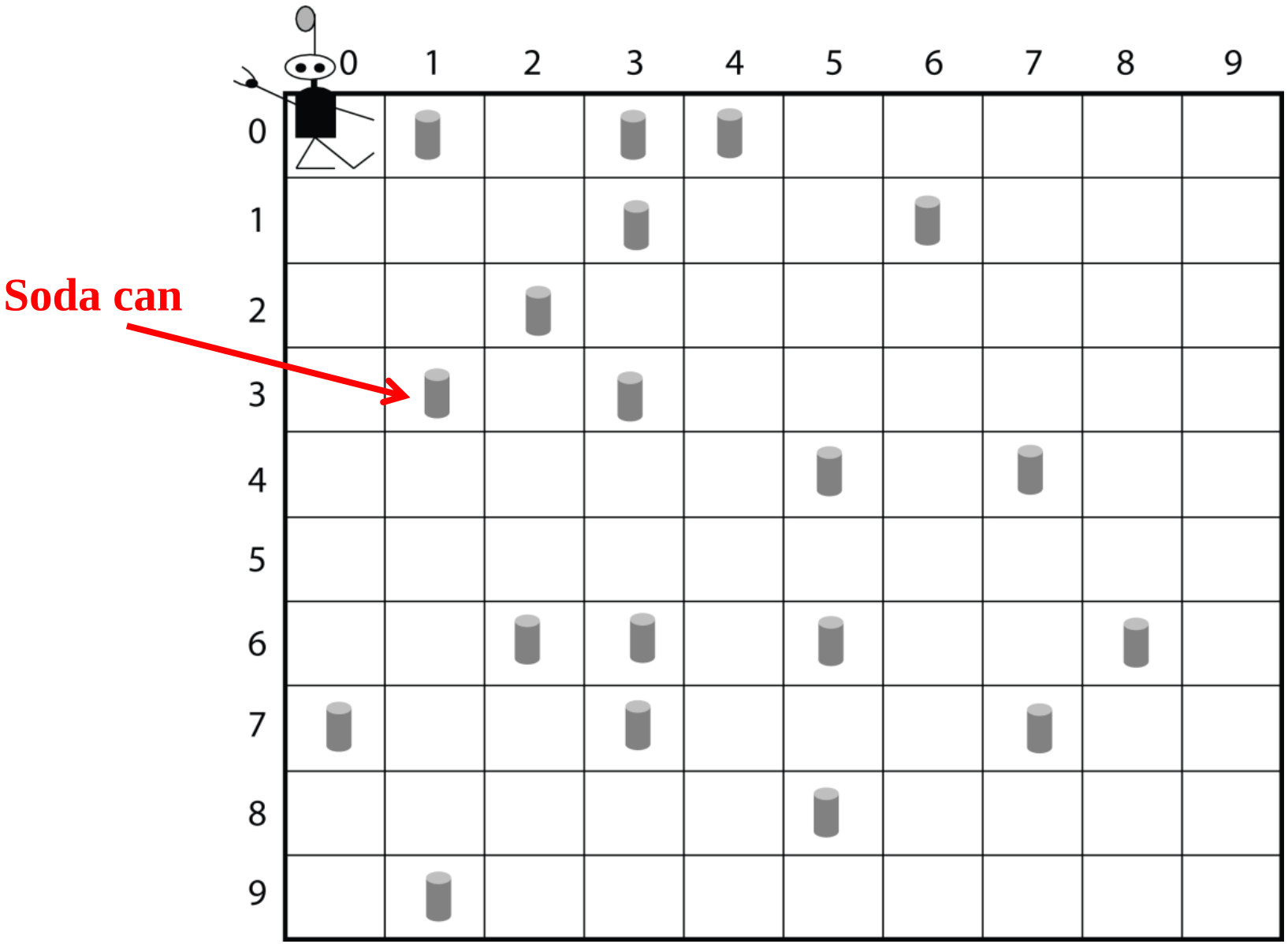
- Begins by generating a random pool of trial solutions:
  - Each solution is a **chromosome**
  - Each component of a chromosome is a **gene**
- Repeatedly generate new pools
  - Each new chromosome is an offspring of two parents from the previous pool
  - Probabilistic preference used to select parents
  - Each offspring is a combination of the parent's genes

Meet Robby

Robby:
The Virtual Soda Can Collecting Robot
(Mitchell, 2009)



Robby's World



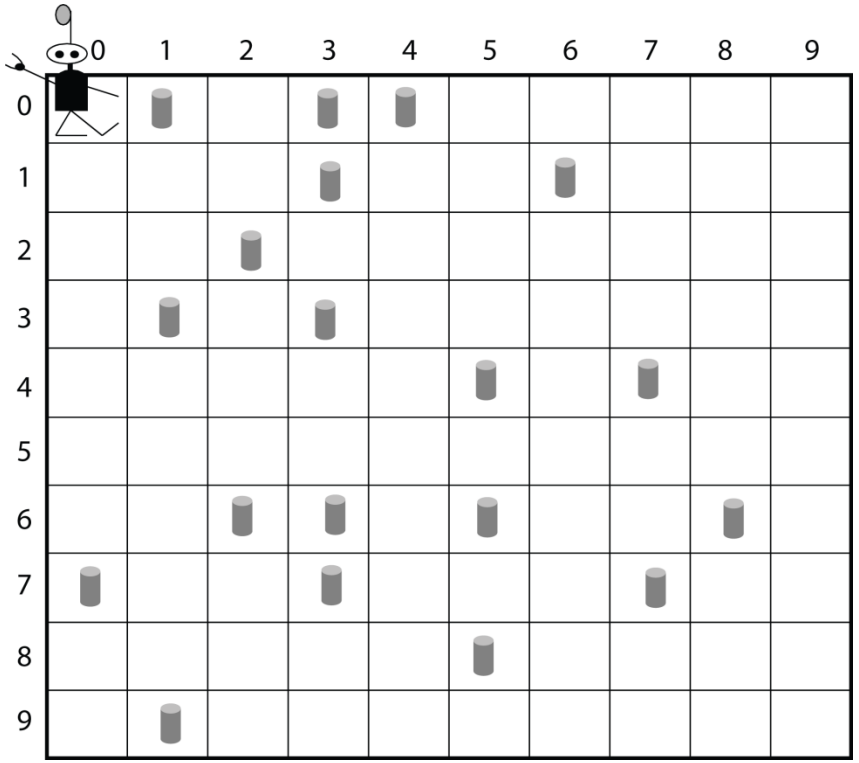
What Robby Can See and Do

Input:
Contents of North, South, East, West, Current

Possible actions:

- Move N
- Move S
- Move E
- Move W
- Move random
- Stay put
- Try to pick up can

Goal: **Use a genetic algorithm to evolve a control program (i.e., *strategy*) for Robby.**



Rewards/Penalties (points):

Picks up can: **10**

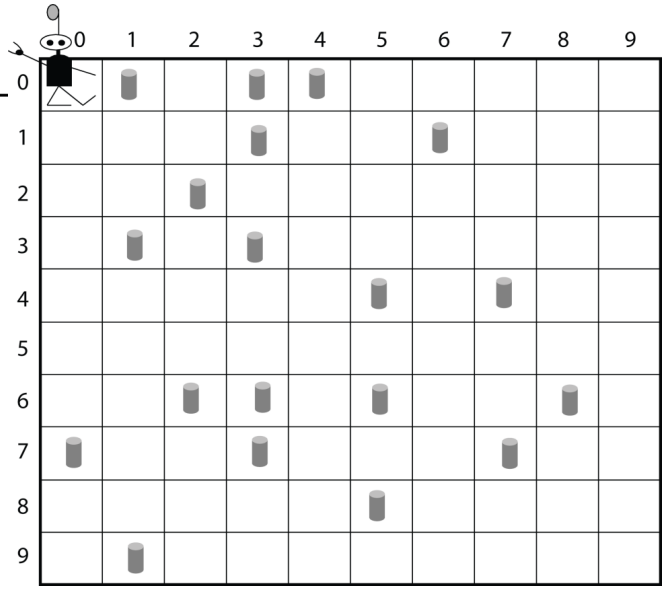
Tries to pick up can on empty site: **-1**

Crashes into wall: **-5**

Robby's Score: Sum of rewards/penalties

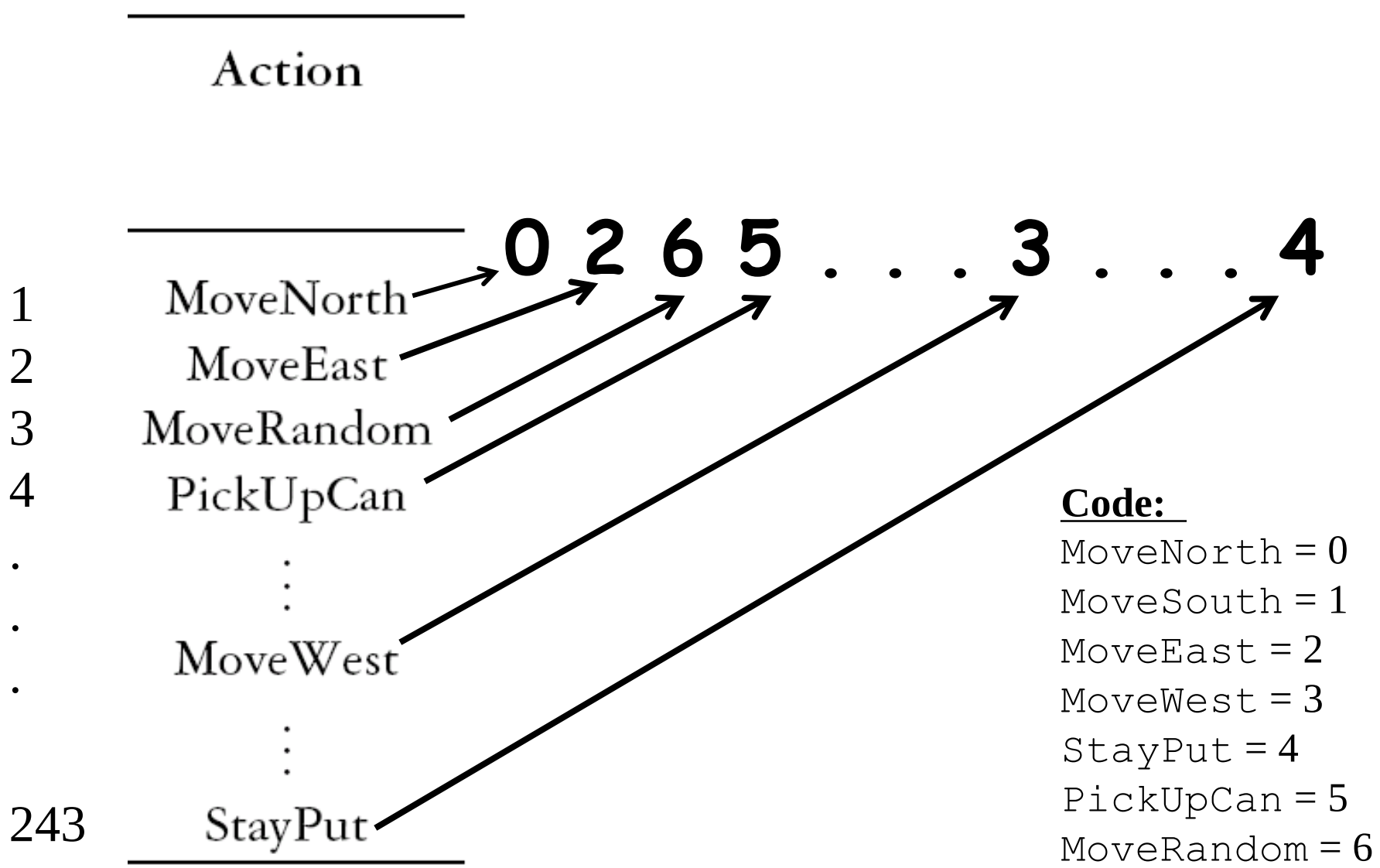
One Example Strategy

	Situation					Action
	North	*South*	*East*	*West*	*Current Site*	
1	Empty	Empty	Empty	Empty	Empty	MoveNorth
2	Empty	Empty	Empty	Empty	Can	MoveEast
3	Empty	Empty	Empty	Empty	Wall	MoveRandom
4	Empty	Empty	Empty	Can	Empty	PickUpCan
.	⋮	⋮	⋮	⋮	⋮	⋮
.	Wall	Empty	Can	Wall	Empty	MoveWest
.	⋮	⋮	⋮	⋮	⋮	⋮
243	Wall	Wall	Wall	Wall	Wall	StayPut



Encoding a Strategy

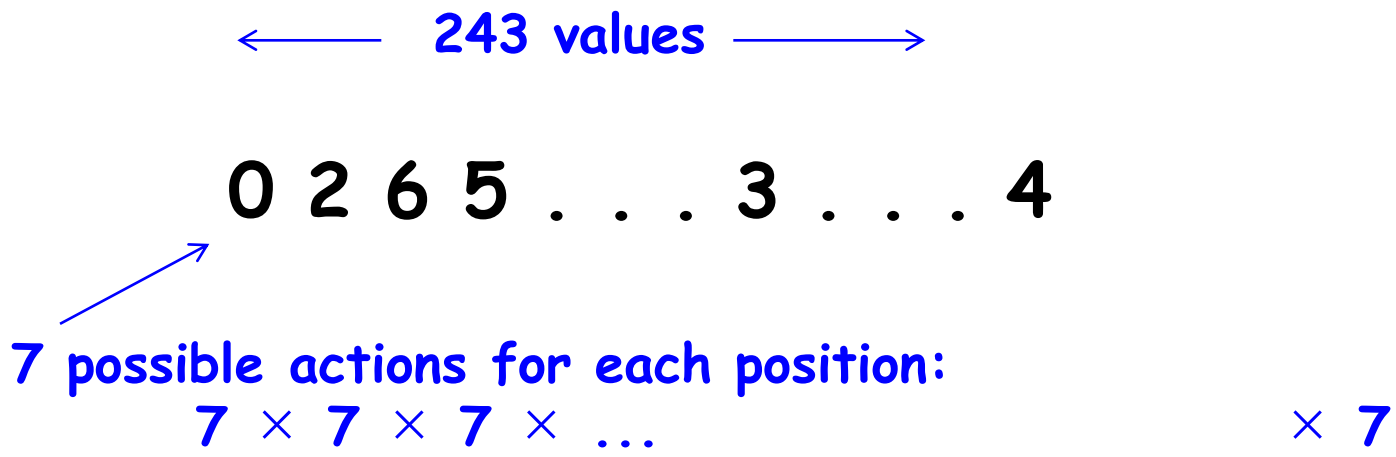
	Situation						Action
	North	*South*	*East*	*West*	*Current Site*		
1	Empty	Empty	Empty	Empty	Empty	1	MoveNorth
2	Empty	Empty	Empty	Empty	Can	2	MoveEast
3	Empty	Empty	Empty	Empty	Wall	3	MoveRandom
4	Empty	Empty	Empty	Can	Empty	4	PickUpCan
.	⋮	⋮	⋮	⋮	⋮	.	⋮
.	Wall	Empty	Can	Wall	Empty	.	MoveWest
.	⋮	⋮	⋮	⋮	⋮		⋮
243	Wall	Wall	Wall	Wall	Wall	243	StayPut


Action
0 2 6 5 . . . 3 . . . 4
1
2
3
4
.
.
.
243
MoveNorth
MoveEast
MoveRandom
PickUpCan
MoveWest
StayPut
Code:
MoveNorth = 0
MoveSouth = 1
MoveEast = 2
MoveWest = 3
StayPut = 4
PickUpCan = 5
MoveRandom = 6

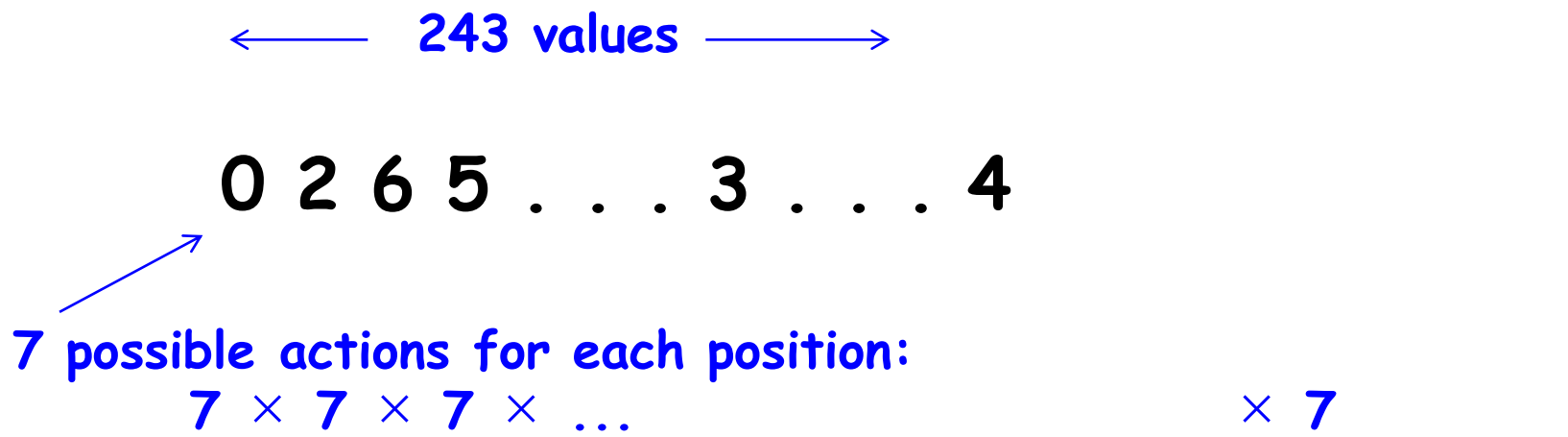
Question: How many possible strategies are there in our representation?

0 2 6 5 . . . 3 . . . 4

Question: How many possible strategies are there in our representation?



Question: How many possible strategies are there in our representation?



Goal: Have GA search intelligently in this vast space for a good strategy

1. Generate 200 Random Strategies

Individual 1:
233003234216303435305460061025625151141622604356543340665115141565022064064205100664321616152165202236443336334601332650300040622050243165006111305146664232401245633345524126143441361020150630642551654043264463156164510543665346310551646005164

Individual 2:
164113431210253603403612414312011042354625253042020445164336656103532215310513144062212061463143215461025652364442202534034530502005620634026331002453416430151631210012214400664012665246351650154123113132453304433212634555005314213064423311000

Individual 3:
204233444024112261321364526324642122061221222526606261444361253251266406133534015341111020616422665314552254023405115503130222020065445125062206631426135532010000400031640130154160162006134440626160505641421553133236021503355131253632642630551

.
.
.

Individual 200:
346325251360010122256121060433011352051553201306560053222350433242506412425526553463534552305332661201063212455442344061365430246240160663016464641103026540006334126150352262106063624260550616616344255124354464110023463330440102533212142402251

2. Calculate Fitness of Each Strategy

```
def calcFitness(robby):
  total_reward = 0

  num_envs = 200
  num_moves_per_env = 100

  for i in range(num_envs):
    env = generate_new_env()
    for j in range(num_moves_per_env):
      total_reward += perform(robby, env)

  return total_reward / num_envs
```

3. Select Two Topmost Strategies

Parent 1:

164113431210253603403612414312011042354625253042020445164336656103532215310513144062212061463143215461025652364442202534034530502005620634026331002453416430151631210012214400664012665246351650154123113132453304433212634555005314213064423311000

Parent 2:

204233444024112261321364526324642122061221222526606261444361253251266406133534015341111020616422665314552254023405115503130222020065445125062206631426135532010000400031640130154160162006134440626160505641421553133236021503355131253632642630551

4. Cross Them Over and Make a New Child

Parent 1:

1641134312102536034036124143120110423546252530420204516433665
6103532215310513144062212061463143215461025652364442202534034 5
3050200562063402633100245341643015163121001221440066401266524 6
3516501541231131324533044332126345550053142130644233110 00

Parent 2:

2042334440241122613213645263246421220612212225266062614443612 5
3251266406133534015341111020616422665314552254023405115503130 2
2202006544512506220663142613553201000040003164013015416016200 6
1344406261605056414215531332360215033551312536326426305 51

Child:

1641134312102536034036124143120110423546252530420204516433665
6103532215310513144062212061463143215461025652364442202534034 5
3050200562063402633100245613553201000040003164013015416016200 6
1344406261605056414215531332360215033551312536326426305 51

5. Randomly Mutate a New Child

Parent 1:

1641134312102536034036124143120110423546252530420204451643366561035322153105131440622120614631432154610256523644422025340345305020056206340263310024534164301516312100122144006640126652463516501541231131324533044332126345550053142130644233110000

Parent 2:

2042334440241122613213645263246421220612212225266062614443612532512664061335340153411110206164226653145522540234051155031302220200654451250622066314261355320100004000316401301541601620061344406261605056414215531332360215033551312536326426305510

Mutate to “0”

Child:

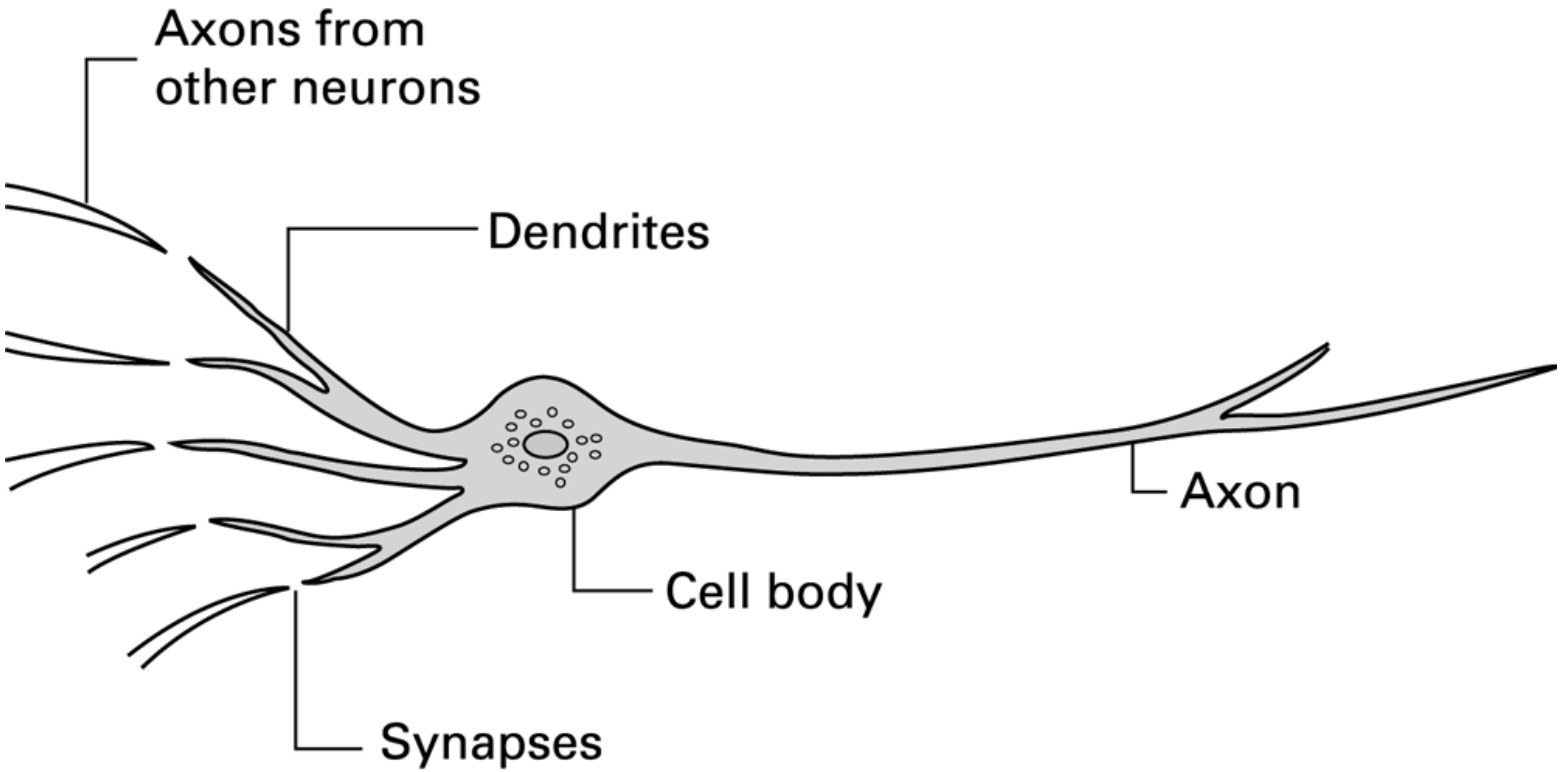
1641134312102536034036124143120110423546252530420204451643366561035322153105131440622120614631432154610256523644422025340345305020056206340263310024561355320100004000316401301541601620061344406261605056414215531332360215033551312536326426305510

Mutate to “4”

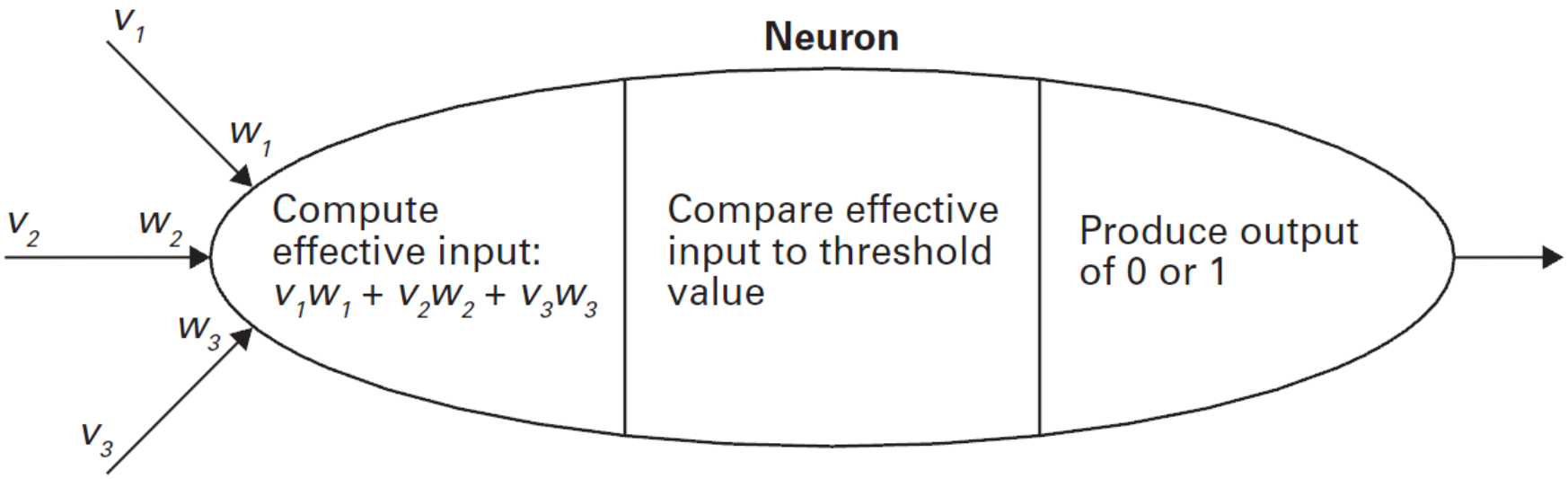
Artificial Neural Networks

- Artificial Neuron
  - Each input is multiplied by a weighting factor.
  - Output is 1 if sum of weighted inputs exceeds the threshold value; 0 otherwise.
- Network is programmed by adjusting weights using feedback from examples.

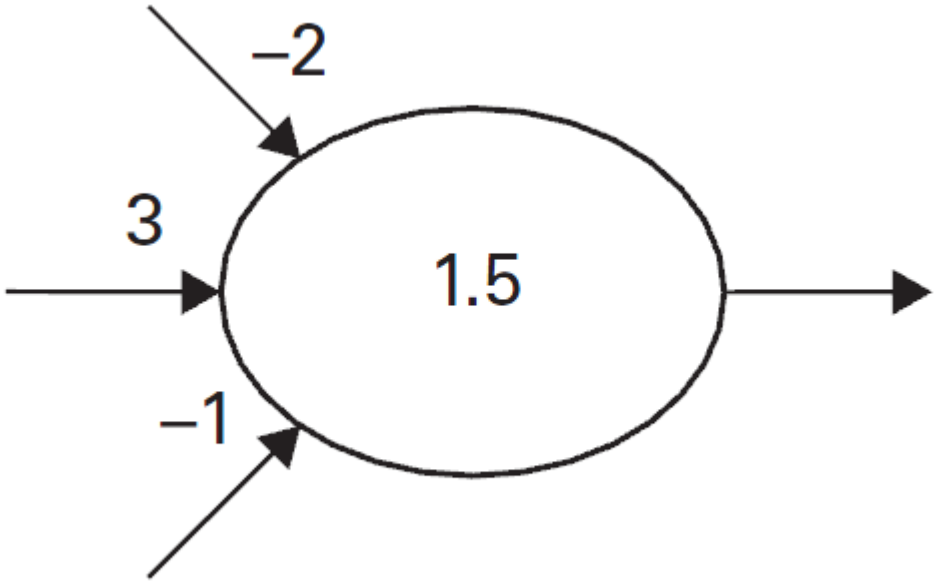
A neuron in a living biological system



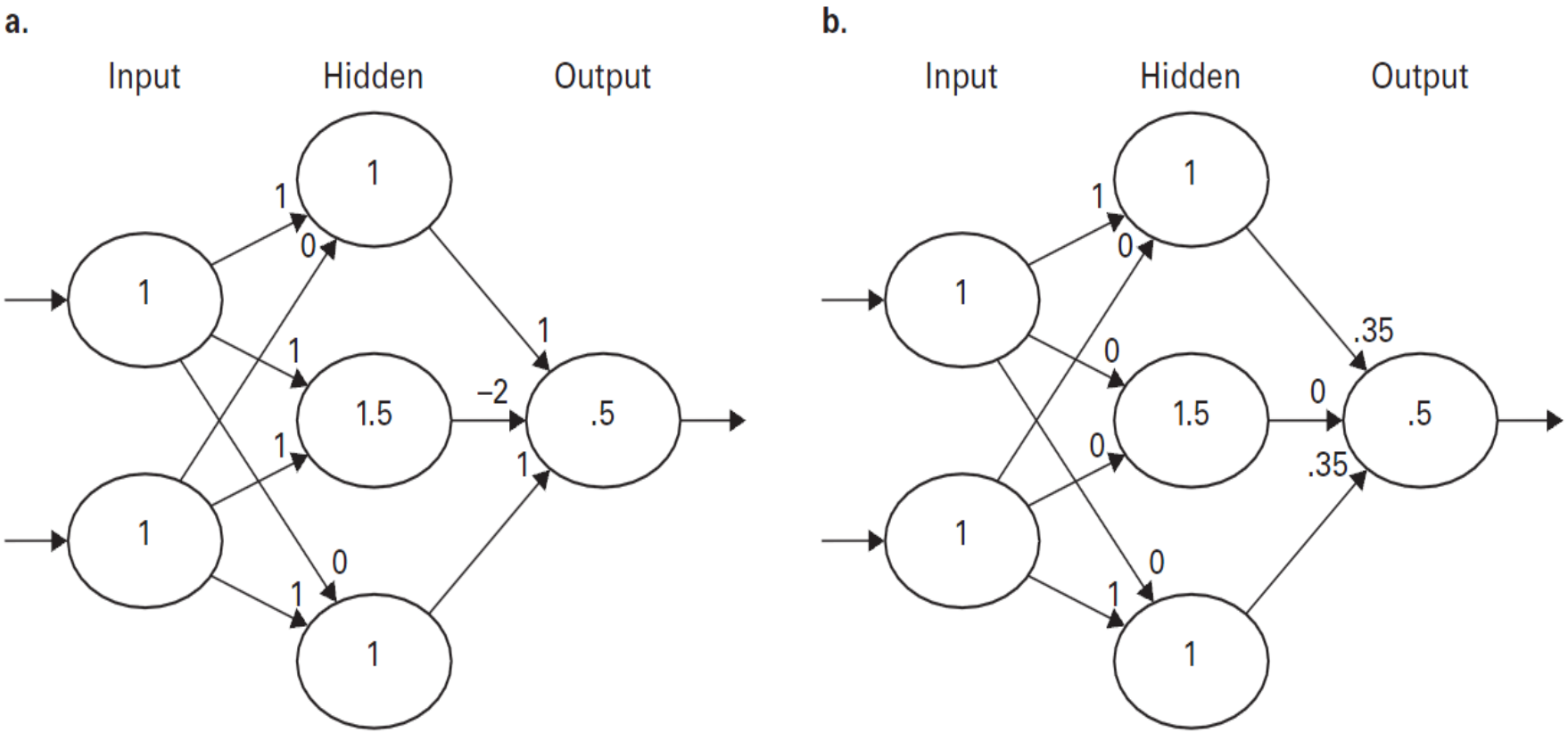
The activities within a processing unit



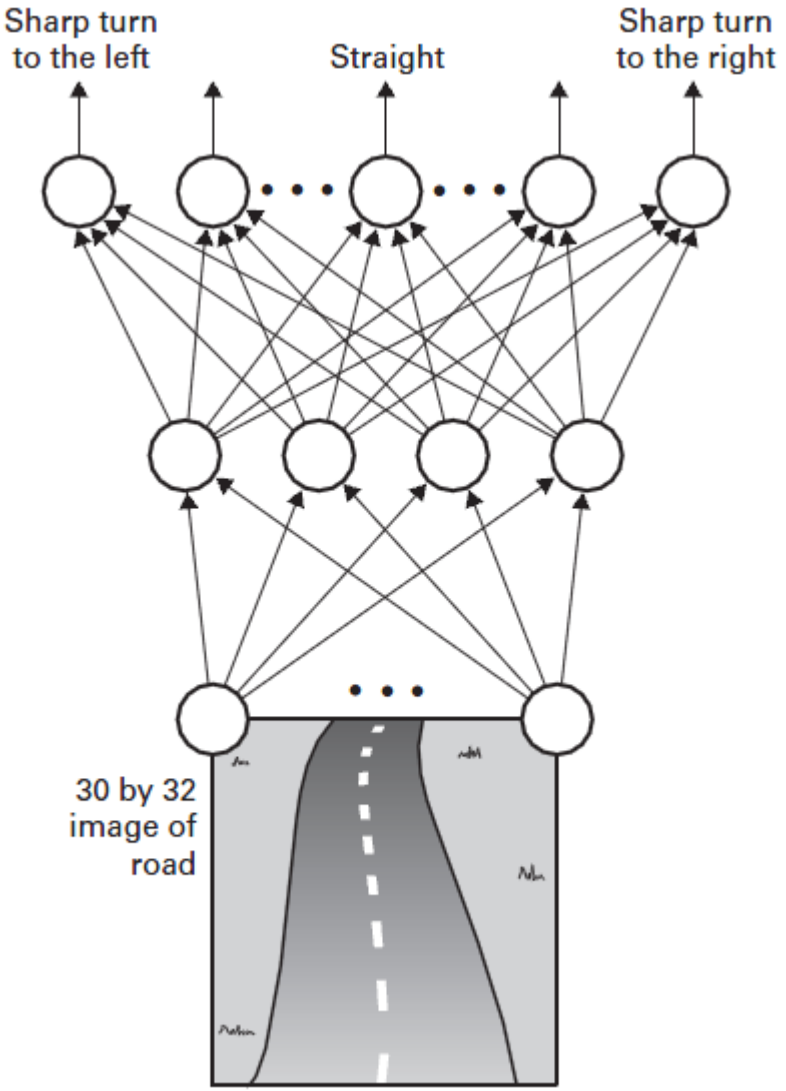
Representation of a processing unit



A neural network with two different programs



The structure of ALVINN

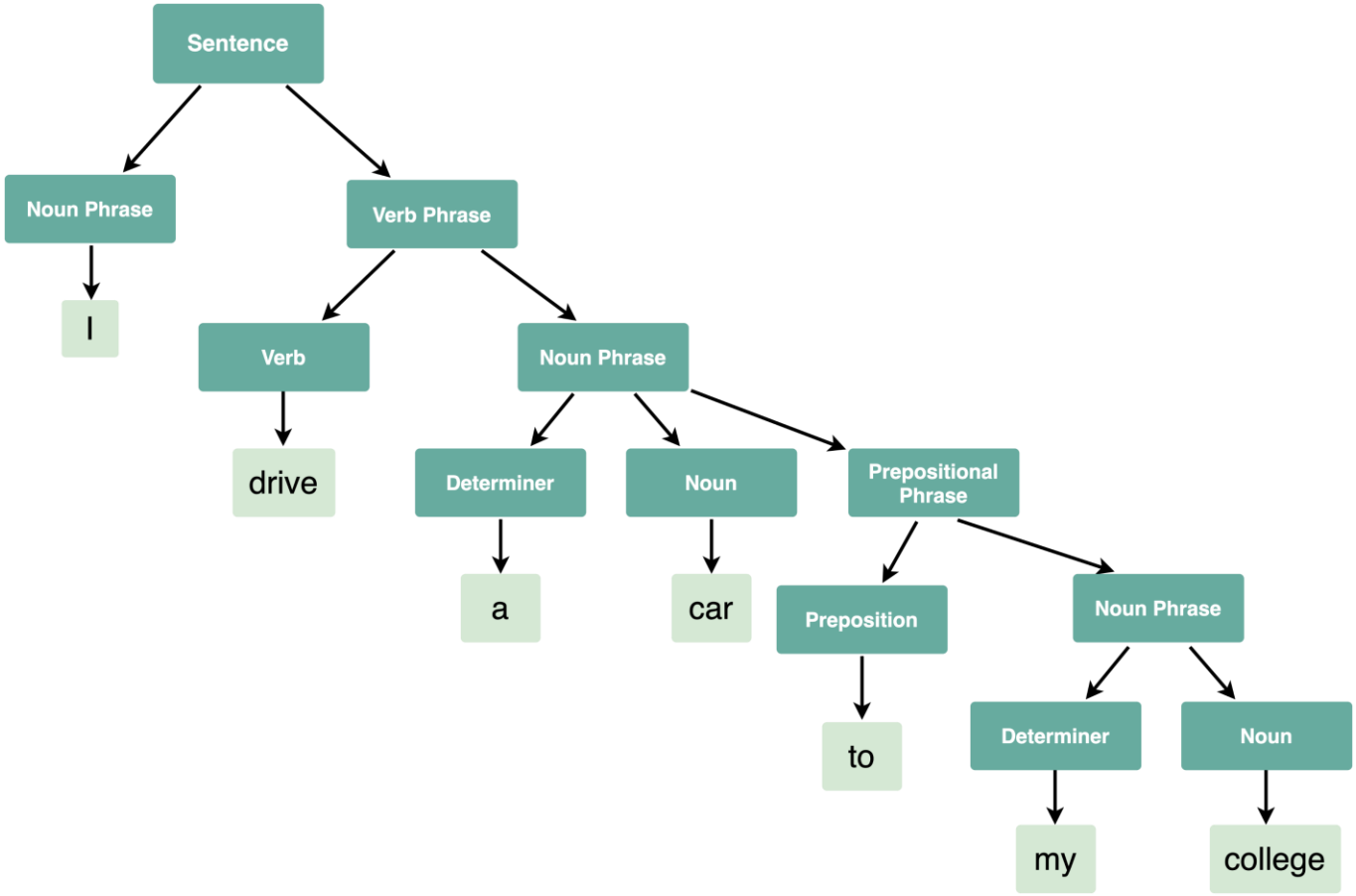


Turing Test

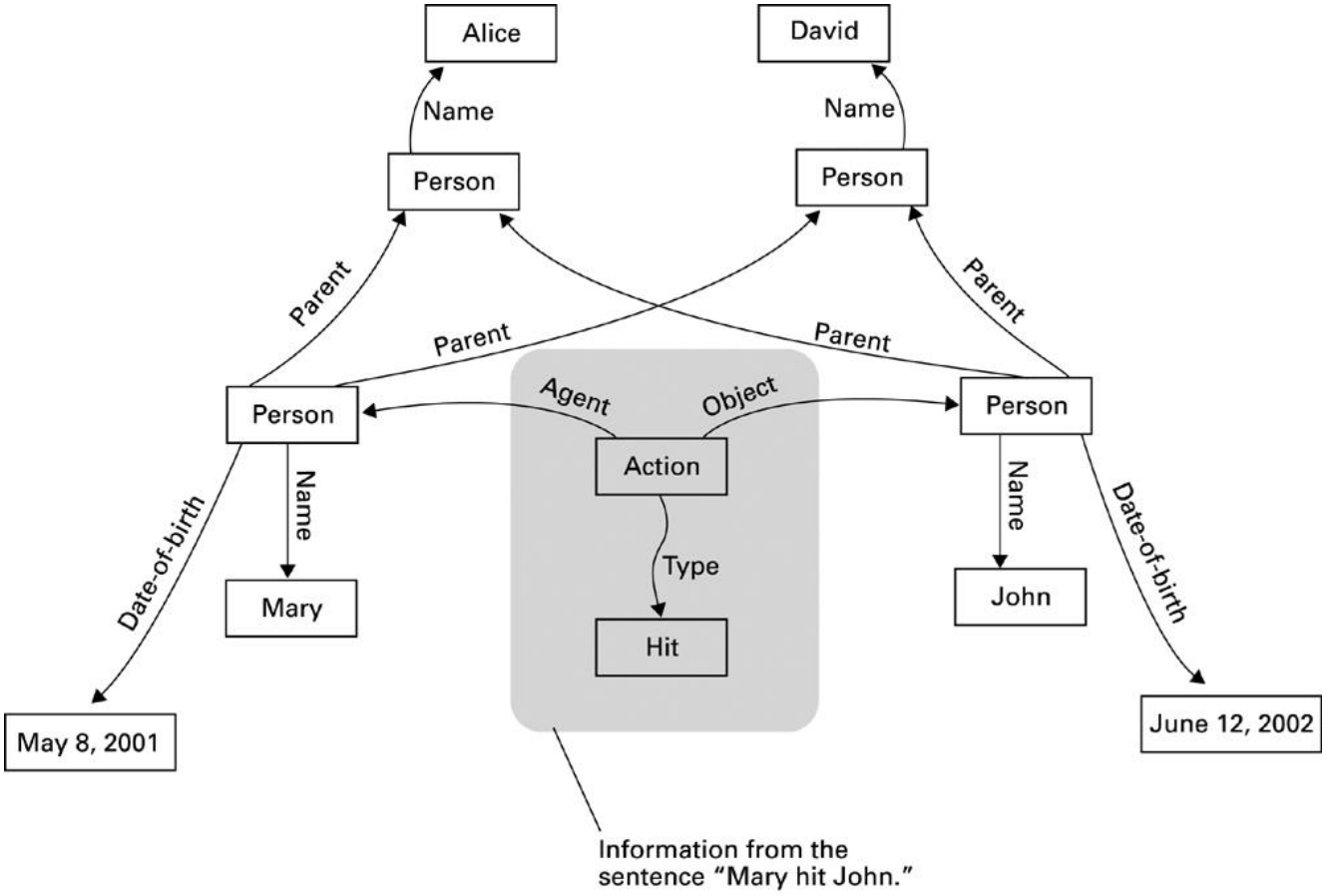
- Proposed by Alan Turing in 1950, a benchmark for progress in artificial intelligence
  - Test setup: human interrogator communicates with test subject by typewriter.
  - Test goal: can the human interrogator distinguish whether the test subject is human or machine?

- Examples
  - The DOCTOR program created by Joseph Weizenbaum in 1960s may fool some naive human interrogator; similar to today's MSN robots
  - Today, CAPTCHA are used to stop web-bots

Natural Language Processing: Understanding Syntax



Natural Language Processing: Understanding Semantics



Natural Language Processing: Understanding Context

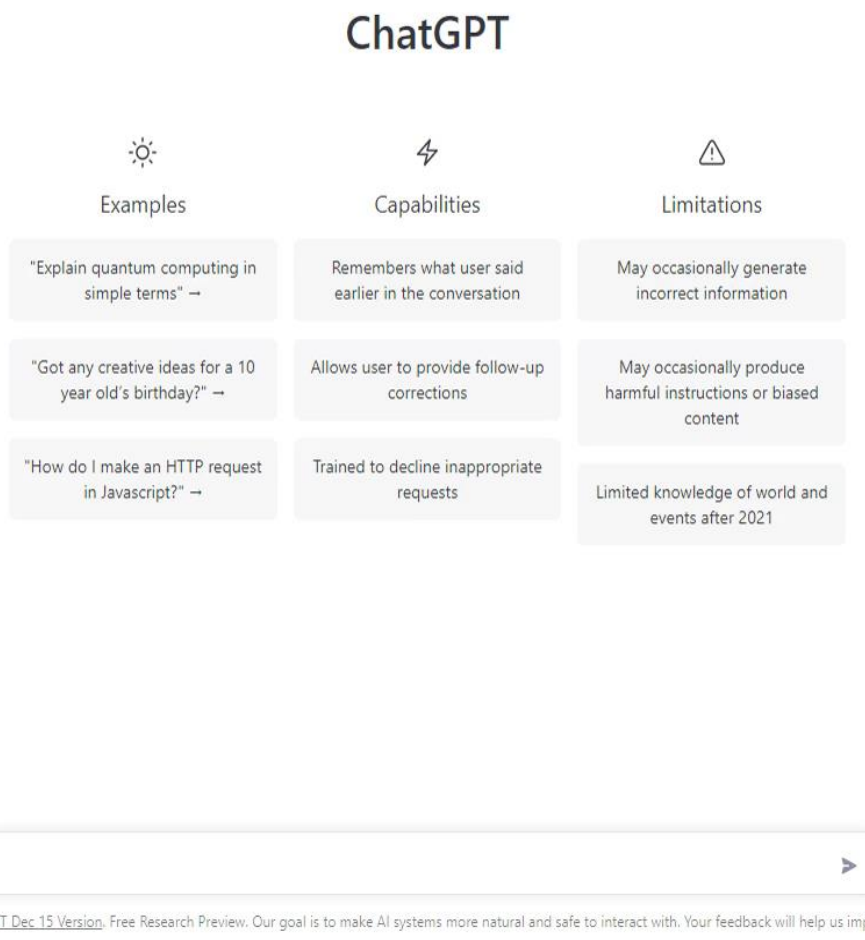
I poured tea from teapot to cup till it is full. What is it?

It sounds like you are describing the process of pouring tea from a teapot into a cup until the cup is full. This is a common action that people often do when they want to enjoy a cup of tea. The tea that you pour from the teapot into the cup is called "brewed tea," and it is made by infusing dried tea leaves in hot water. The teapot is a container specifically designed for brewing and serving tea, and the cup is a small, shallow vessel that is used for drinking tea or other beverages.

I poured tea from teapot to cup till it is empty. What is it?

It sounds like you are describing the process of pouring all of the tea from the teapot into the cup until the teapot is empty. This means that you have transferred all of the brewed tea from the teapot into the cup, and there is no more tea left in the teapot. The tea that you poured from the teapot into the cup is called "brewed tea," and it is made by infusing dried tea leaves in hot water. The teapot is a container specifically designed for brewing and serving tea, and the cup is a small, shallow vessel that is used for drinking tea or other beverages.

ChatGPT


ChatGPT
Examples
"Explain quantum computing in simple terms" →
"Got any creative ideas for a 10 year old's birthday?" →
"How do I make an HTTP request in Javascript?" →
Capabilities
Remembers what user said earlier in the conversation
Allows user to provide follow-up corrections
Trained to decline inappropriate requests
Limitations
May occasionally generate incorrect information
May occasionally produce harmful instructions or biased content
Limited knowledge of world and events after 2021
ChatGPT Dec 15 Version. Free Research Preview. Our goal is to make AI systems more natural and safe to interact with. Your feedback will help us improve.

Issues Raised by Artificial Intelligence

- Is programming relevant anymore?
- Is any intellectual work relevant anymore?
- If a computer can do a job better than a human, when should a human do the job anyway?
- When should a computer's decision be trusted over a human's?
- What would be the social impact if computer "intelligence" surpasses that of many humans?

Suggested Activity/Reading

- Depth-first/Breadth-first Searches in Python:
  - http://eddmann.com/posts/depth-first-search-and-breadth-first-search-in-python/
- AI Textbook Author and Google's Head of Research - Peter Norvig's homepage - tonnes of puzzles, interesting problems and research topics in AI:
  - http://norvig.com
- Neural Networks and Deep Learning by Michael Nielsen - loads of examples in vanilla Python:
  - http://neuralnetworksanddeeplearning.com/